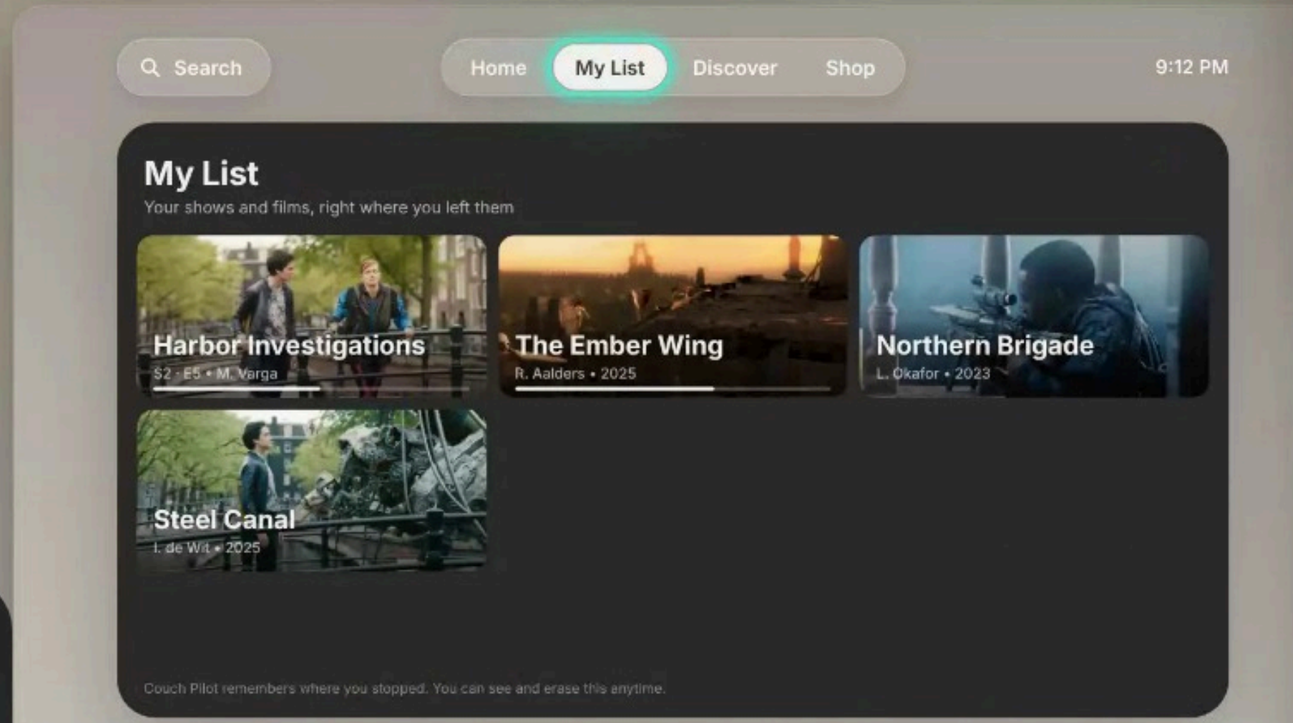
Couch Pilot

The self-driving remote.



"Resume my detective show where I left off, and turn it up."
One sentence. Four steps. Zero clicks.

BUILD, SHIP, SHAPE · AMAZON DEVELOPER HACKATHON 2026 · SUBMISSION

Couch Pilot — the self-driving remote

The self-driving remote. Say one sentence and Couch Pilot plans every step and drives your Fire TV while you watch. Alexa+ understands, Couch Pilot acts.

Primary track	Alexa+: self-hosted MCP server (spec 2025-11-25, Streamable HTTP) + simulated Alexa+ experience (real MCP client)
Also runs on	Fire OS: Android TV app (Kotlin WebView, D-pad, ☰ = talk, Back = stop, TTS) · prebuilt APK in `dist/`
Live demo (no install)	couchpilot.netlify.app · click “try a demo” or press M
Code (public, MIT)	github.com/chessoren/couchpilot
Demo video (2:32)	<add your public YouTube/Vimeo link>
Run it in 2 minutes	`npm install && npm start` → TV at `localhost:8787`, Alexa+ simulator at `/alexa`, MCP at `/mcp`
Proof it works	`npm test` (planner) · `npm run test:e2e`: a real MCP client drives a real TV page through the four demos, 11/11 checks pass
Project status	New: built entirely during the submission window · Entrant: France
Content	Fictional catalog · artwork and clips from Blender Foundation open movies (CC BY) · no third-party brands, no music

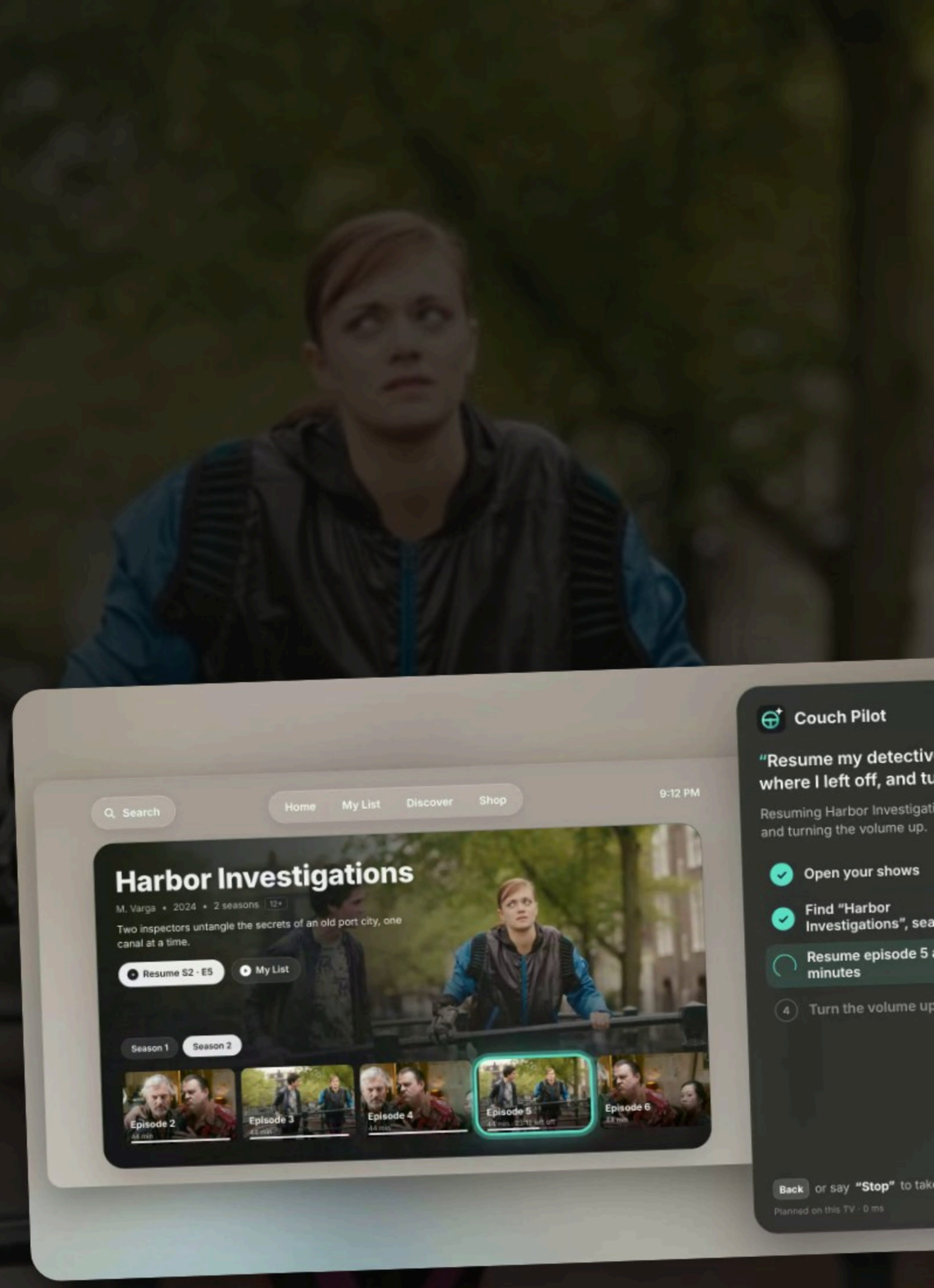
Couch Pilot

The self-driving remote.

Say the goal. It drives the TV.

Alexa understands what you say. Couch Pilot does what you want.

Alexa+ · MCP 2025-11-25 | Fire TV · Fire OS | Accessibility



EVERY EVENING STARTS WITH A CHORE

Watching one episode takes 20+ button presses.

Turn on · find the app · pick a profile · type the title letter by letter · season · episode · subtitles · volume. Each step is easy. **The chain is exhausting**, and one wrong button sends you back.

For older adults

Menus change after every update. Many wait for a relative to "put the TV back".

For people with low vision

A wall of thumbnails is almost unusable, even with a screen reader.

For shaky or painful hands

Aiming at tiny buttons dozens of times a night hurts.

For them it's not an annoyance. It's a wall.

DESIGNED FOR MIREILLE FIRST. LOVED BY EVERYONE.

Mireille, 78

Lives alone, low vision, arthritis. Loves detective shows. Afraid to "break" the box her daughter installed.

“Resume my detective show where I left off, and turn it up.”

She no longer needs to understand the menus.

Karim, 34 · low vision

“Find me a documentary about space, under an hour.”

Léa, 38 · hands full

“Put on something for the kids.” Volume capped, stops in 30 min.

Thomas, 27 · comfort

“Movie night.” Film, subtitles, volume, reminder.

Design rule for every decision: **would Mireille understand it, and feel safe?**

ONE GOAL → ONE COMPLETE PLAN, EXECUTED IN FRONT OF YOU

Couch Pilot doesn't replace Alexa+. It gives Alexa+ hands on the TV.

	CLASSIC VOICE ASSISTANT	COUCH PILOT
Unit of work	One command	**A whole goal**
Actions per sentence	One	**Many, chained**
Visibility	You don't see what was understood	**Plan on screen, each step checks off live**
Ambiguity	Guesses or fails	**One short question with choices**
Taking over	Grab the remote	**“Stop” or Back, at any moment**
Memory	Little context	**Where you stopped, your routines**

THE SAME 5-BEAT LOOP, EVERY TIME

1

Listen

Mic button or wake word. “I'm listening.”

2

Understand

Your sentence appears, word for word.

3

Show the plan

Numbered steps, before anything moves.

4

Drive

The TV navigates by itself. Steps check off.

5

Confirm

“All set. Enjoy.” Then it gets out of the way.

You always know

what was understood · what will happen · what is happening

You can always

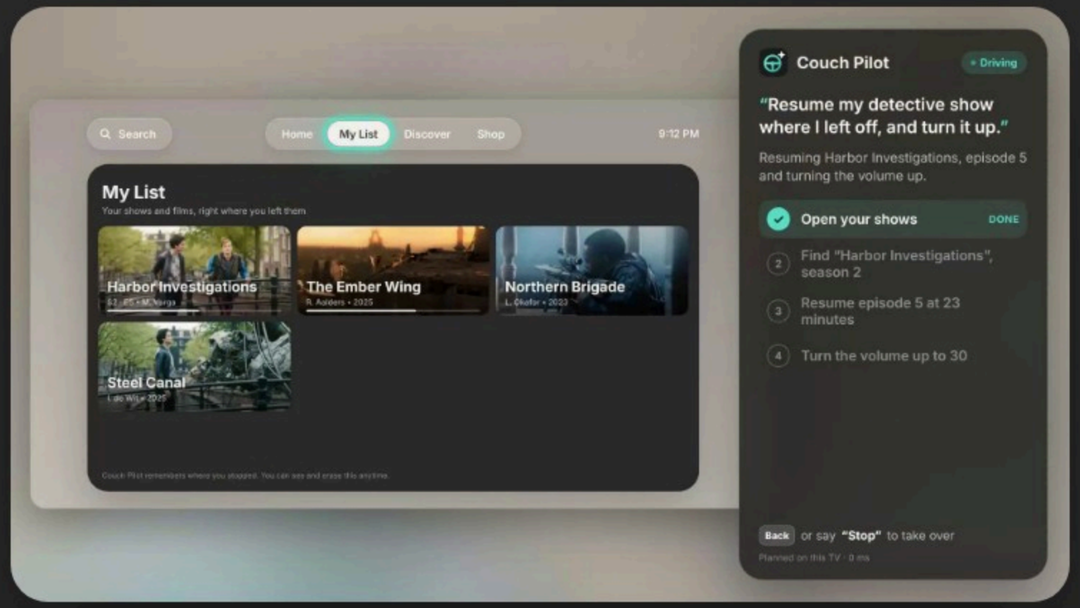
stop it with one word or one button — and you're never surprised by a payment

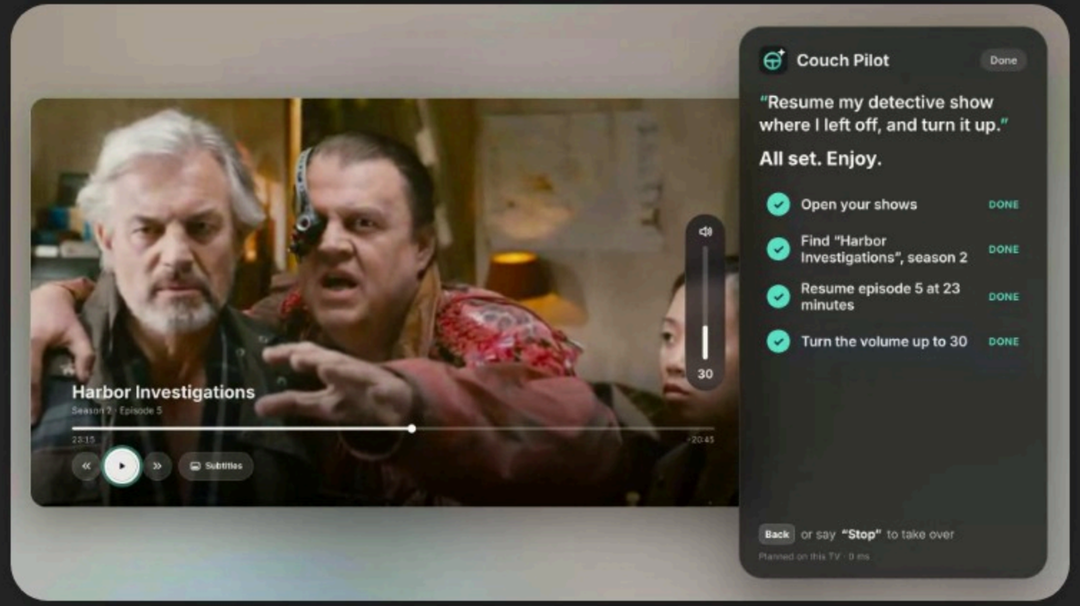
ONE SENTENCE

Four steps. Zero clicks.

“Resume my detective show where I left off, and turn it up.”

- Open your shows
- Find “Harbor Investigations”, season 2
- Resume episode 5 at 23 minutes
- Turn the volume up to 30

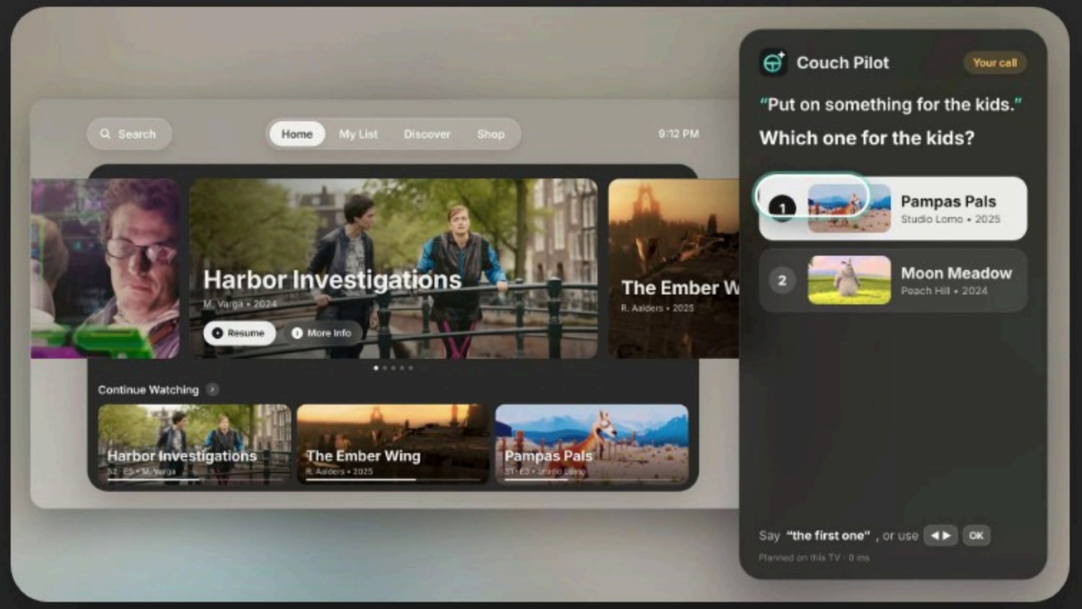


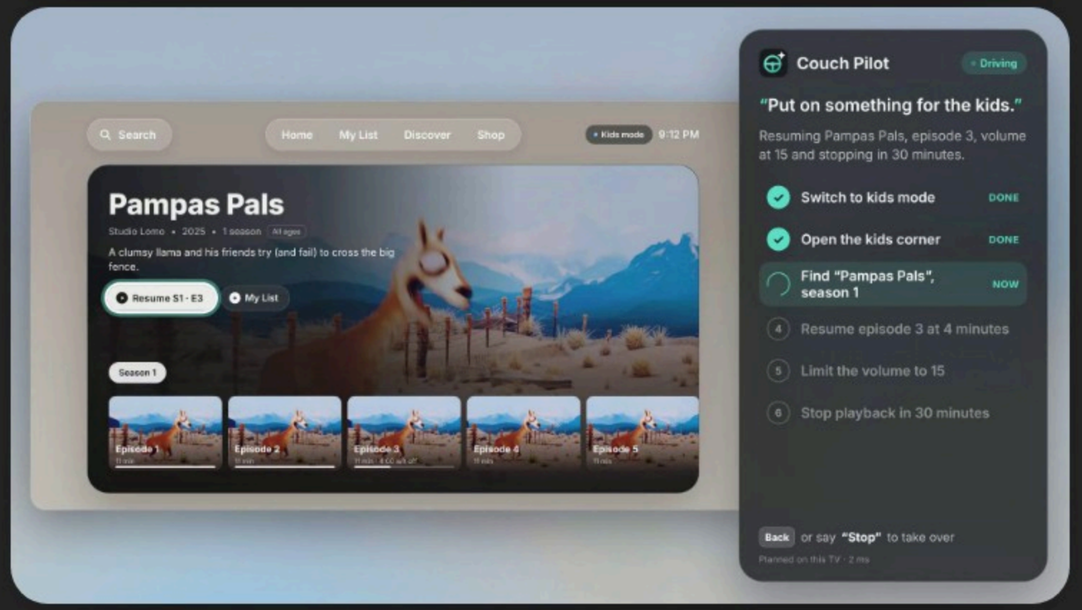


VOICE + REMOTE + SCREEN

When it's not sure, it asks. One question, never more.

“Put on something for the kids.” Two kids shows fit, so two big numbered tiles appear. Answer **“the first one”**, say the name, or just press ▼ **OK** on the remote. The agent finishes alone: kids mode, volume capped at 15, stops in 30 minutes.



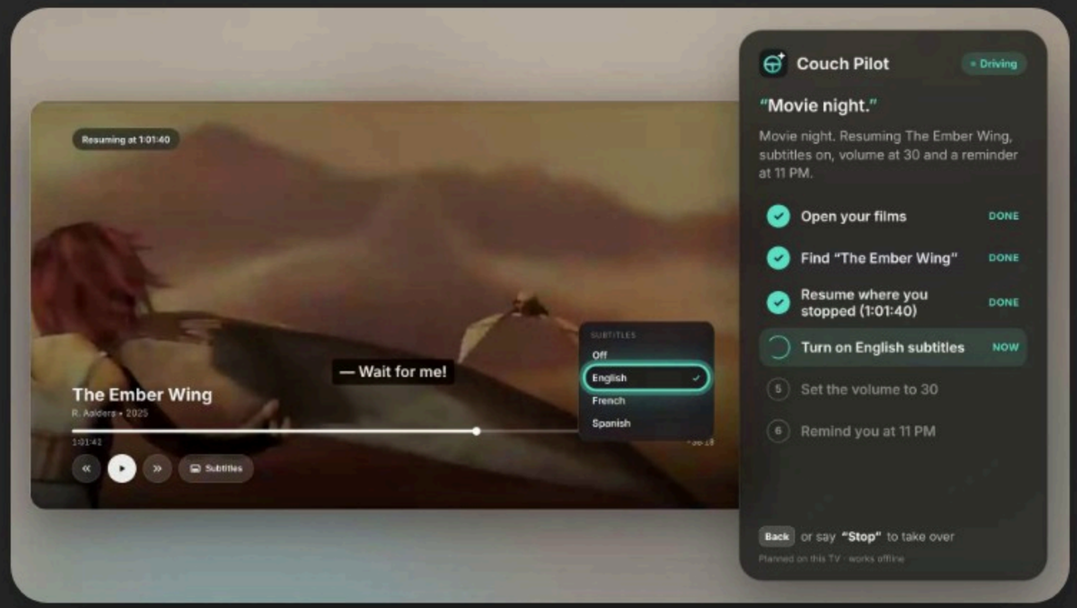


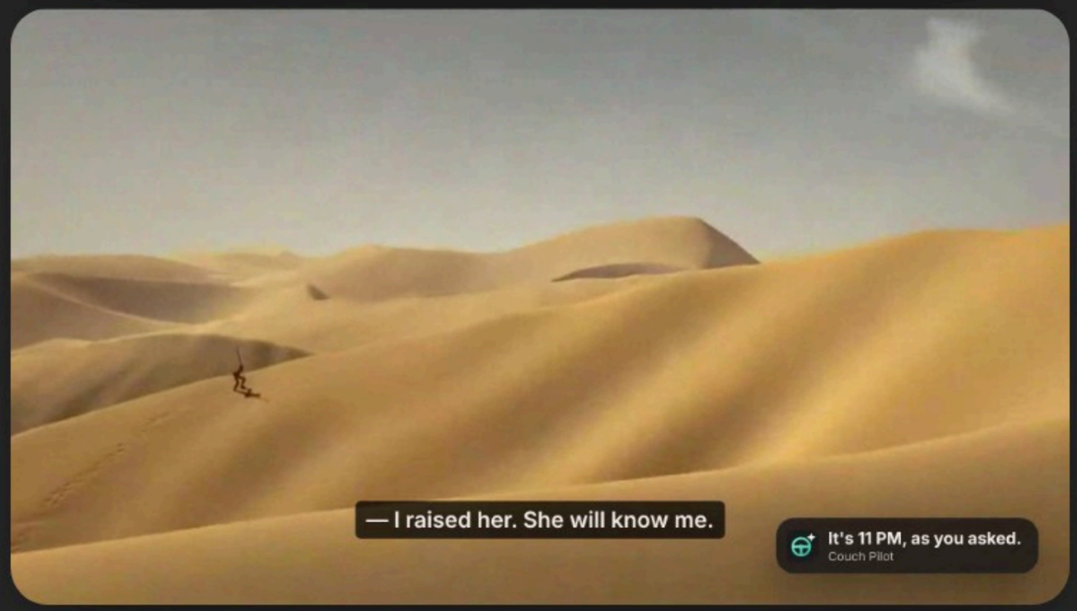
TWO WORDS

“Movie night.”

Resume yesterday's film where you stopped · turn on your usual subtitles · set your usual volume · remind you at 11 PM.

Also built in: **Kids mode**, **Quiet mode**, **Same as yesterday**. The household memory (where you stopped, your preferences) is visible and erasable.



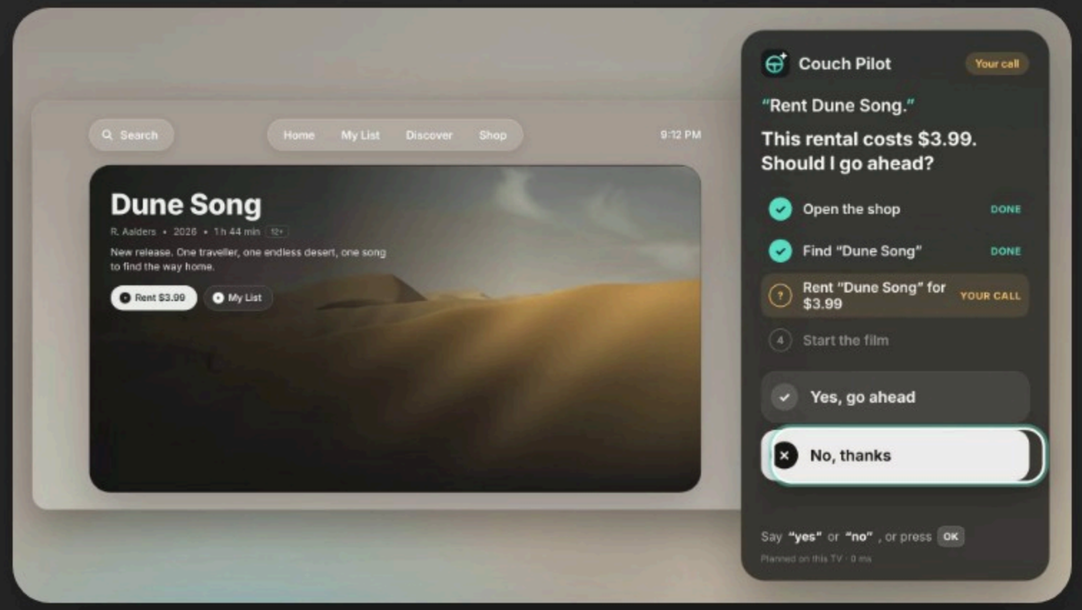


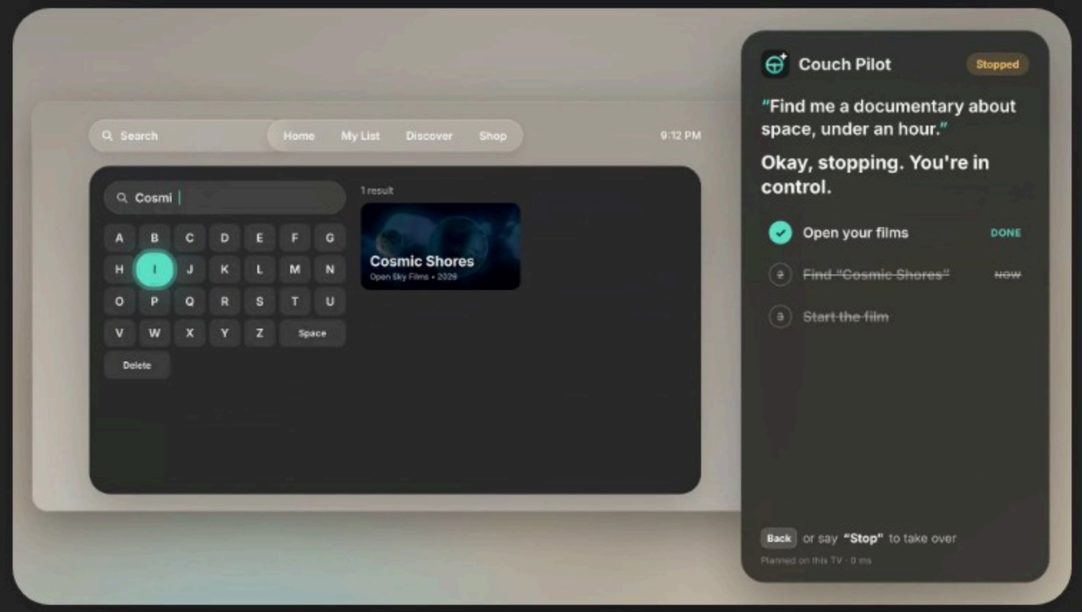
YOU'RE ALWAYS IN CONTROL

Money needs a yes. One word stops it.

"Rent Dune Song." → **"This rental costs $3.99. Should I go ahead?"** Default focus is on **No**: an accidental OK never spends money.

Say **"Stop"**, press **Back**, or touch any remote key: the agent stops instantly, marks the remaining steps as skipped and says **"You're in control."** It never resumes on its own.





THREE TRUST LEVELS — ENFORCED IN CODE, NOT IN A PROMPT

LEVEL	EXAMPLES	WHAT COUCH PILOT DOES
Free	Open, search, play, volume, subtitles, seek, reminders, sleep timer	**Acts alone**, showing every step
Confirm	Rent or buy, power off, irreversible changes	**Shows the price or consequence**, waits for a clear yes
Forbidden	Passwords, PIN codes, payment details, parental controls	**Refuses politely**, hands back control

Any planner, same rules

On-device planner or an Alexa+ agent over MCP: every plan passes `enforceGuardrails()`. No caller can downgrade a risk level.

Kids stay kids

In kids mode, grown-up titles are refused by voice and the time limit can't be lifted by voice.

ALEXA+ UNDERSTANDS → COUCH PILOT ACTS → THE FIRE TV EXECUTES

Alexa+

Conversation, intent.

Alexa+ simulator in the repo: a real MCP client (official SDK).

Couch Pilot MCP server

Spec **2025-11-25** · **Streamable HTTP** · stateless.

Tools: **couchpilot_run_goal**, **couchpilot_answer**, **couchpilot_stop**, tv_get_state + 7 primitives. Resources: catalog, guardrails.

TV bridge

Server-Sent Events down, HTTP results up. Questions return as **needs_input**.

Fire TV app

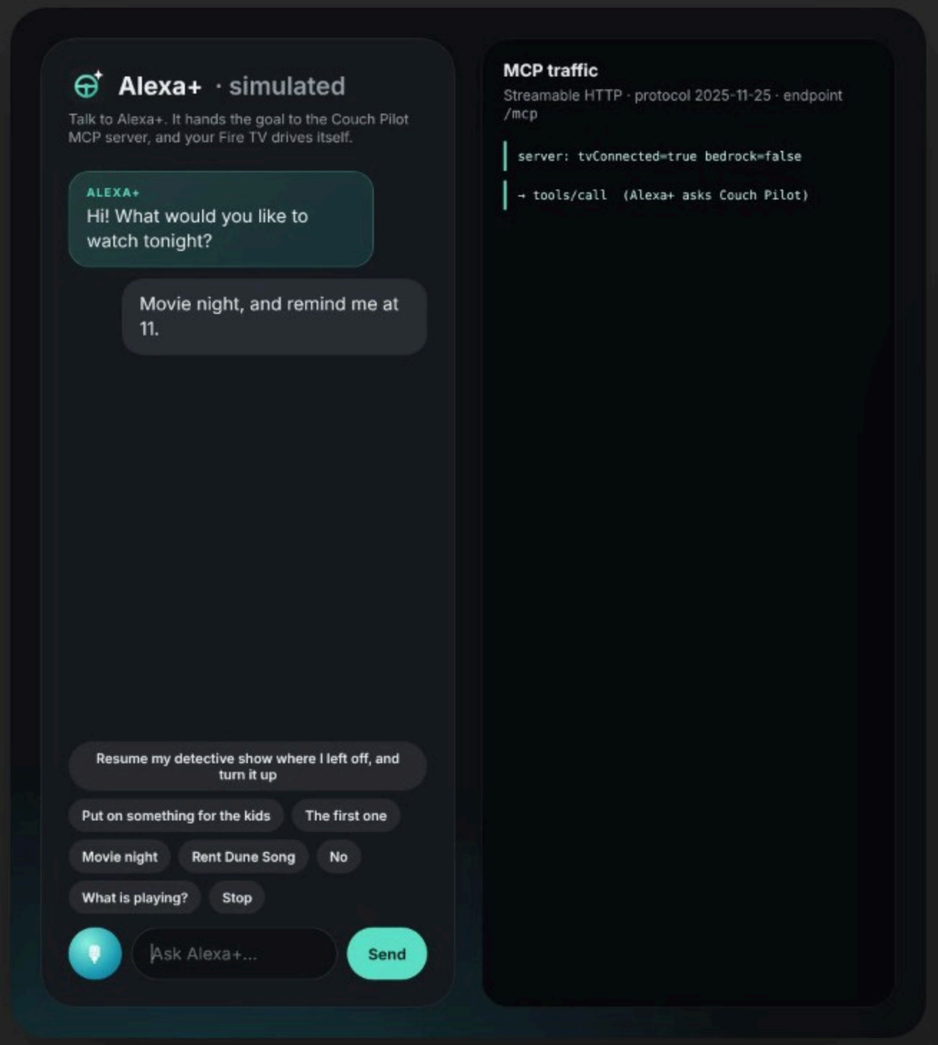
Fire OS APK (Kotlin WebView) · D-pad · Android speech + TTS. Planner → executor drives the real UI.

Who is the AI?

Alexa+ understands the user. **Couch Pilot** plans the TV steps, enforces the guardrails and drives the screen. On the TV, an **on-device planner** works offline for the remote's mic button.

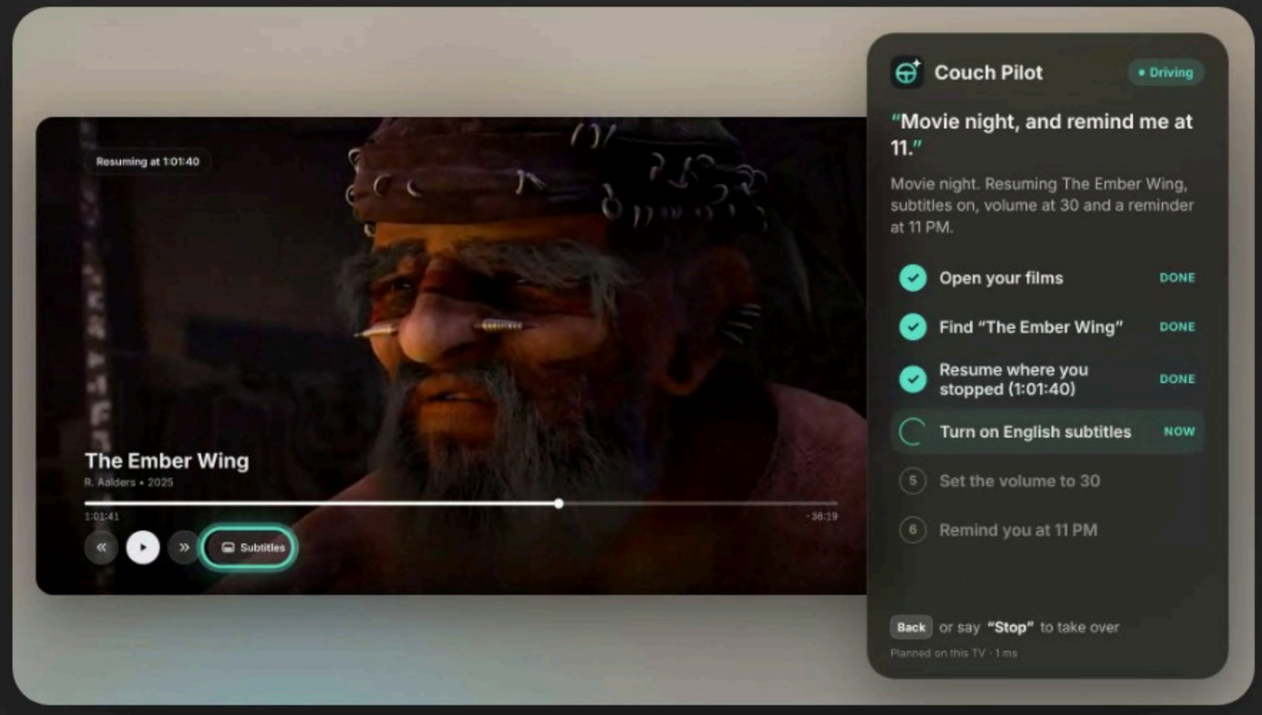
One codebase

Plain ES modules, no build step. Same app in the browser, in the Fire OS APK, and behind MCP.



A REAL ROUND TRIP

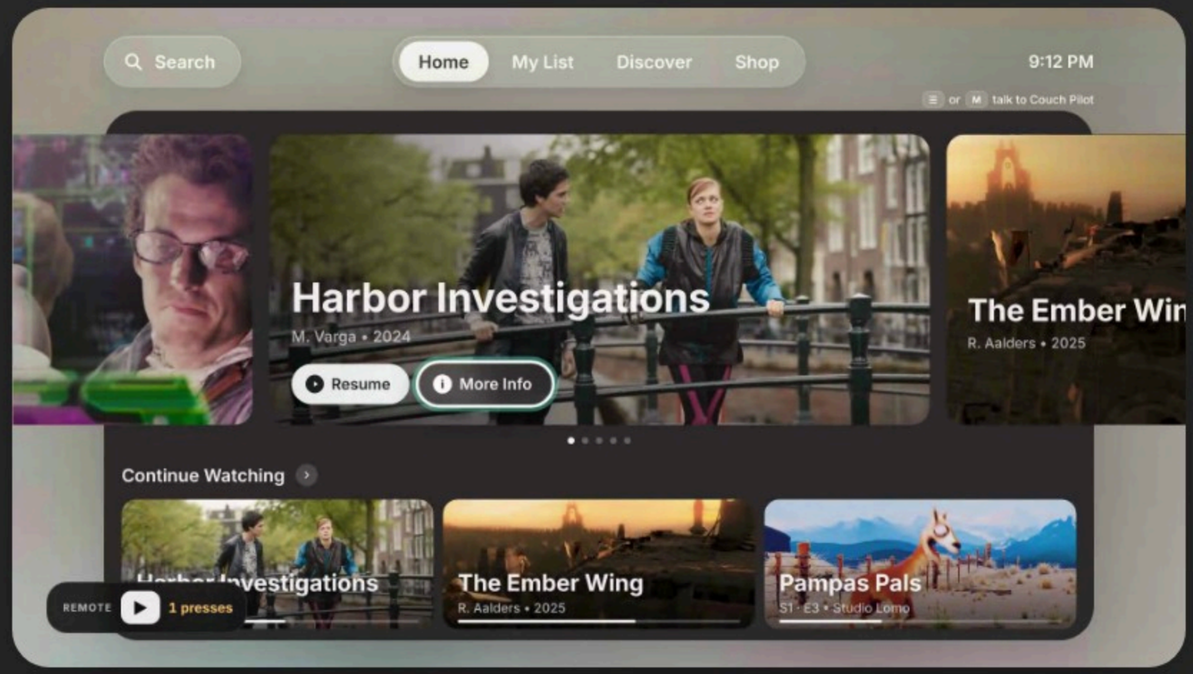
Alexa+ (simulated) → `tools/call couchpilot_run_goal` → the TV drives itself → the result comes back to Alexa+ to be spoken.



Covered by an automated end-to-end test: a real MCP client drives a real TV page through all four demos (11/11 checks pass).

READABLE AT 3 METERS

1. Big type, strong contrast, ≤ 5 lines at once
2. One accent color — and never color alone: ✓ + “Done”
3. The TV shrinks left: you **see** it drive
4. One focus ring glides from target to target
5. Numbered choices: answer “the two” without seeing
6. Disappears when the video plays



Glass pills, a dark rounded window, hero carousel that bleeds past the edges: a calm 10-foot UI where the agent panel is the only addition.

IMPACT

Autonomy: Mireille no longer waits for her daughter's visit.
Less isolation: a TV you can use keeps you connected.
Less fatigue: every press avoided counts.
For Amazon: the simplest TV on the market for seniors and people with disabilities; Alexa+ that acts, not just answers.

ROADMAP

V2 household voices · full read-aloud accessibility mode · user-made routines (“When I say Good night...”) · remote help from a daughter's phone.

V3 Ring: “Who's at the door?” as picture-in-picture · Bee: a “decompress” routine after a long day · Echo lights dim for movie night.

Our ask to Amazon

An official, consent-based way for third-party assistants to navigate Fire TV apps on the user's behalf. With it, Couch Pilot becomes an accessibility layer for every Fire TV.

The remote has fifty buttons.
Couch Pilot needs only one:
the one that lets you speak.

github.com/chessoren/couchpilot · MIT · **npm install && npm start**

Fictional catalog. Footage from Blender Foundation open movies (CC BY). No third-party brands.

ABOUT THE PROJECT

The story

Inspiration

Mireille is 78. She lives alone, her eyesight is fading and arthritis makes her hands hurt. She loves detective shows. Since her daughter installed a new streaming box, she hasn't dared touch the settings: the menus change after every update, the icons are tiny, and one wrong button sends her somewhere she doesn't recognize. Most evenings she waits for a visit so someone can "put the TV back the way it was."

Mireille is a composite persona, but you probably know someone like her. Watching one episode with subtitles takes more than twenty button presses: open the app, pick a profile, type the title letter by letter with the arrows, choose the season, the episode, open the audio menu, set the volume. Each step is easy. The chain is exhausting. For older adults, people with low vision or people with motor impairments, it isn't an annoyance. It's a wall.

Voice assistants helped, but they still work one command at a time, and you can't see what they understood or what they're doing. The real problem isn't talking to the TV. It's handing the TV a whole task and trusting it to finish the job.

What it does

Couch Pilot is **the self-driving remote**. You say a goal in plain language:

> "Resume my detective show where I left off, and turn it up."

Couch Pilot turns the sentence into a **plan**, shows the plan on screen, and **drives the TV itself** while you watch: open your shows → find *Harbor Investigations*, season 2 → resume episode 5 at 23 minutes → volume to 30. Each step lights up and checks off in real time. Then it says "All set. Enjoy." and disappears. **One sentence. Four steps. Zero clicks.**

- **It asks instead of guessing.** "Put on something for the kids" matches two shows, so it asks one question with two big numbered tiles. Answer by voice ("the first one") or with the remote (▼ OK): **voice, remote and screen in one flow**. It then finishes alone: kids mode, volume capped at 15, playback stops after 30 minutes.
- **Routines in two words.** "Movie night" resumes yesterday's film where you stopped, turns on your usual subtitles (you watch it open the subtitle menu), sets your usual volume and reminds you at 11 PM. Also: Kids mode, Quiet mode, Same as yesterday.
- **You're always in control.** Anything that costs money or can't be undone needs a clear yes ("This rental costs $3.99. Should I go ahead?"), with focus defaulting to **No** so an accidental OK never spends money. Saying **"Stop"**, pressing **Back** or touching any remote key stops it instantly: "Okay, stopping. You're in control." Passwords, payment details and parental controls are refused outright.
- **Alexa+ gets hands.** Couch Pilot is an **MCP server (spec 2025-11-25, Streamable HTTP)**. An Alexa+ agent can call `couchpilot_run_goal("movie night")` and the Fire TV drives itself. Questions come back as `needs_input`, so the user always gives the final yes. **Alexa+ understands what you say. Couch Pilot does what you want.**

How we built it

- **The TV app** (`web/`): a 10-foot UI on a fixed 1920×1080 stage (glass pills, dark rounded window, hero carousel bleeding past the edges, Continue Watching), with a spatial **D-pad focus engine** and a single focus ring that glides from target to target. Plain ES modules with no build step, so the same code runs in a browser, inside the Fire OS app and behind MCP.

- **The agent** (`web/js/planner.js`, `web/js/pilot.js`): a planner that splits a sentence into clauses (play, volume, subtitles, seek, reminders, sleep timers, rentals), resolves the target from the household memory ("my detective show", "the film from yesterday"), expands routines and asks when two titles fit. The **executor drives the real UI with the same primitives as the remote** (focus, press, type on the on-screen keyboard), which is what makes it visibly "self-driving". It retries a failed step once, then hands back control with a clear message.
- **Guardrails in code, not in a prompt:** an action registry with three trust levels (free / confirm / forbidden) and `enforceGuardrails()`, applied to every plan whoever produced it: on-device planner, an Alexa+ agent over MCP, or any other caller. No caller can downgrade a risk level.
- **Alexa+ track:** `server/mcp.js` uses the official **MCP TypeScript SDK** (protocol `2025-11-25`, stateless Streamable HTTP): 4 high-level tools (`couchpilot_run_goal`, `couchpilot_answer`, `couchpilot_stop`, `tv_get_state`), 7 primitives (`tv_play_title`, `tv_set_volume`, `tv_set_subtitles`, `tv_seek`, `tv_navigate`, `tv_schedule`, `tv_rent_title`), 2 resources (catalog, guardrails) and tool annotations. A **TV bridge** (Server-Sent Events down, HTTP POST up) turns every tool call into visible steps on the TV. The **simulated Alexa+ experience** (`/alexa`) is a real MCP client that chooses the tool, relays questions and speaks the answer, with the MCP traffic shown live.
- **Fire TV:** `android/` is a Kotlin **Fire OS / Android TV** app: full-screen WebView serving the bundled app through `WebViewAssetLoader`, leanback launcher banner, D-pad, ☰ Menu = talk, Back = stop, Android `SpeechRecognizer` when available, and the system **TextToSpeech** so the agent speaks. A prebuilt APK is in `dist/`.
- **Testing:** unit tests for the planner, plus an **end-to-end test where a real MCP client drives a real TV page in headless Chromium** through all four demos (11/11 checks pass). The demo video was recorded from the real app with Playwright. Nothing is mocked.
- **Content:** a fictional catalog with artwork and clips from Blender Foundation open movies (CC BY), credited in the repo. No third-party brands, no music.

Challenges we ran into

- **Making it visibly self-driving.** Executing commands instantly would have looked like magic and felt like a black box. We rebuilt the executor around remote-control primitives with readable pacing, and shrink the TV to the left while the agent drives so you can watch it work.
- **Multi-turn flows over a voice protocol.** A tool call that needs the user's choice or a payment confirmation can't just block. We designed a portable `needs_input` + `couchpilot_answer` pattern, and the user can also answer on the TV with the remote, in the same flow.
- **Interrupting safely.** Stop had to work from anywhere: voice, Back, any remote key, or MCP, and in the middle of a step. Every step checks an abort signal between micro-actions, and the agent never resumes on its own.
- **Testing on a TV without a TV.** The Fire TV/Vega simulator needs hardware virtualization our cloud build machine didn't have, and the remote's voice key is reserved for Alexa. We built a Fire OS WebView app (mapped to ☰ Menu), recorded the same app in the browser, and wrote it all down in our friction log.

Accomplishments that we're proud of

- A multi-step agent that acts **visibly** and stays **interruptible** at every moment.
- One flow that mixes voice, remote and screen without breaking.
- A real MCP integration with an automated end-to-end test, not a mock-up.
- A design a 78-year-old with low vision can follow from the couch: big type, colour never used alone (✓ + "Done" + accent), numbered choices she can answer without seeing.

What we learned

Trust is the product. An agent acting on your behalf is only acceptable if you can see what it understood, see what it's about to do, watch it do it, and stop it with one word. And for accessibility, fewer button presses beat more features.

What's next for Couch Pilot

- Testing with a dozen older adults and people with low vision, to tune the language and the panel.
- A full read-aloud accessibility mode, household voice profiles, user-made routines ("When I say Good night, do this, this and this"), remote help so a daughter can set up her mother's routines from her phone.
- The wider Amazon ecosystem: Ring ("Who's at the door?" as picture-in-picture without leaving the movie), Bee (a "decompress" routine after a long day), Echo lights dimming for movie night.
- **Our ask to Amazon:** an official, consent-based way for assistants to navigate Fire TV apps on the user's behalf. With it, Couch Pilot becomes an accessibility layer for every Fire TV.

The remote has fifty buttons. Couch Pilot needs only one: the one that lets you speak.

REQUIRED

Product feedback

One entry per tool, API or SDK used to build Couch Pilot.

1. Model Context Protocol, spec 2025-11-25, Streamable HTTP (Alexa+ track)

- **Used for:** the Couch Pilot MCP server (`/mcp`): `couchpilot_run_goal` , `couchpilot_answer` , `couchpilot_stop` , `tv_get_state` , 7 primitives, 2 resources, tool annotations, stateless Streamable HTTP with JSON responses.
- **What worked well:** stateless mode (one server + transport per request) is simple to host and to put behind a load balancer. Tool annotations (`readOnlyHint` , `destructiveHint`) map naturally to our trust levels. `structuredContent` gives clients machine-readable results next to the spoken text.
- **What needs work:** for voice assistants, the spec doesn't say much about long-running tools and "ask the user, then continue" flows. Elicitation exists, but whether Alexa+ supports it is undocumented, so we used a portable `needs_input` + answer-tool pattern.
- **Onboarding:** under an hour from zero to a tool called by a client.
- **Build with it again?** Yes. It's the cleanest way to give Alexa+ new abilities without writing a skill per intent.

2. MCP TypeScript SDK (`@modelcontextprotocol/sdk` 1.32.1)

- **Used for:** `McpServer` + `StreamableHTTPServerTransport` (server), `Client` + `StreamableHTTPClientTransport` (Alexa+ simulator and end-to-end test).
- **What worked well:** `registerTool` with Zod schemas, `LATEST_PROTOCOL_VERSION = 2025-11-25` out of the box, a working client in the same package, so we could test the server for real.
- **What needs work:** the stateless (no session ID) setup is only explained in a code comment in `streamableHttp.d.ts` ; it deserves a top-level README example. Zod 3/4 compatibility is accepted but not explained.
- **Onboarding:** smooth.
- **Build with it again?** Yes.

3. Alexa+ (simulated experience path)

- **Used for:** the "Alexa+ understands, Couch Pilot acts" flow. `web/alexa.html` + `server/alexa-sim.js` simulate the Alexa+ conversation and call Couch Pilot through a real MCP client.
- **What worked well:** the rules explicitly allow the simulated path, which let us focus on the agent and the TV experience.
- **What needs work:** we could not connect a self-hosted MCP server to a real Alexa+ account (see friction log #3). A developer sandbox would let teams prove the real integration.
- **Onboarding:** the hackathon pages were clear on what's accepted. Device-side testing isn't possible yet.
- **Build with it again?** Yes, with a sandbox it would be our first choice for voice on Fire TV.

4. Fire OS / Android TV (Fire TV track)

- **Used for:** `android/`: a Kotlin app with a full-screen WebView (`androidx.webkit` `WebViewAssetLoader` serving the bundled app), leanback launcher and banner, D-pad handling, Menu key = talk, Back = stop, `SpeechRecognizer` when available, `TextToSpeech` for spoken replies.
- **What worked well:** a standard Android TV project builds with stock Gradle/AGP, and the WebView path lets one codebase serve the browser, the APK and the MCP demo. D-pad events reach the WebView as arrow keys.
- **What needs work:** voice input for third-party apps (friction log #2), no headless simulator (friction log #1). We would also like clear guidance on which video codecs the WebView supports per Fire TV generation (we ship WebM/VP9 and MP4/H.264 sources).
- **Onboarding:** good docs for sideloading with ADB. We wrote the wrapper in under an hour.
- **Build with it again?** Yes.

5. Fire TV starter samples and docs

- **Used for:** reference for 10-foot UI patterns (focus, navigation) and the multi-modal UX priority category. We did not fork a sample: our app is plain ES modules so it runs unchanged in the browser and in the Fire OS WebView.
- **What worked well:** `vega-tv-interfaces-sample` and the multi-TV React Native sample show focus management clearly.
- **What needs work:** a "web app on Fire OS" sample (WebView + D-pad + TTS) would help the many teams who prototype in the browser.
- **Build with it again?** Yes.

6. Amazon Bedrock (tried, not used in the submission)

- **Used for:** an optional cloud planner (`server/bedrock.js`: Converse API with a forced `submit_plan` tool limited to the allow-listed actions). Off by default.
- **What worked well:** the Converse API's tool schema is a good fit for constraining an LLM to a fixed action set. The JS SDK is easy to wire.
- **What needs work:** getting from a role ARN to a working call from a cloud container (friction log #6).
- **Build with it again?** Yes, for a production planner. For this hackathon, Alexa+ is the language AI and Couch Pilot executes, so we did not need a second LLM.

Feature request

- **Feature:** an official, consent-based way for third-party assistants (and accessibility apps) to navigate the Fire TV interface and other apps on the user's behalf: open, search, choose season/episode, play, subtitles, volume. Ideally exposed to Alexa+ as MCP tools.
- **Why it matters:** Couch Pilot proves the experience on its own catalog. With this API it becomes an accessibility layer for every Fire TV user: older adults, people with low vision, people with motor impairments.
- **Priority:** Critical

BONUS

Friction log

Written while building Couch Pilot during the hackathon window (October 23, 2026). One entry per blocker, in the format requested by the organizers.

---

1. Running the Fire TV / Vega simulator in a cloud dev environment

- **Task attempted:** run Couch Pilot on the Fire TV / Vega Virtual Device to record the demo from a headless Linux build machine (cloud container, no GPU).
- **Steps taken:** checked the Vega and Fire TV starter docs, looked for a headless or containerized simulator, checked for `/dev/kvm` in the container.
- **Expected:** a way to boot a Fire TV / Vega simulator headlessly (CI, cloud IDEs, AI coding agents) and capture video.
- **Actual:** the simulator needs hardware virtualization (KVM) and a desktop session; it cannot run in a typical cloud container. No hosted or browser-based preview exists.
- **Severity:** Major
- **Workaround:** built the TV app as a web app with a Fire OS (Android TV) WebView wrapper, compiled the APK in the container, recorded the demo from the same app running in headless Chromium, and kept on-device filming for a real Fire TV.
- **Actionable suggestion:** offer a hosted Vega/Fire TV simulator (or a container image with software rendering) that can run in CI and stream or record its screen. AI coding assistants increasingly work in cloud containers.

2. The remote's microphone button is not available to third-party apps

- **Task attempted:** let the user press the voice button on the Fire TV remote to talk to Couch Pilot.
- **Steps taken:** handled key events in the Android activity; tested `SpeechRecognizer.isRecognitionAvailable()` paths in code.
- **Expected:** an app-level way to receive voice input on Fire TV (the voice key, or a system speech-to-text intent).
- **Actual:** the voice key is reserved for Alexa, and Android `SpeechRecognizer` is not guaranteed on Fire OS devices.
- **Severity:** Major (core to an accessibility use case)
- **Workaround:** mapped ☰ **Menu** to "talk to Couch Pilot", fell back to the on-screen keyboard when speech recognition is missing, and made **Alexa+ → MCP** the main voice path.
- **Actionable suggestion:** an Alexa+ "hand-off to app" intent: Alexa transcribes, then delivers the utterance to the foreground app that registered for it (with user consent), or a documented speech-to-text intent on Fire OS.

3. Testing a self-hosted MCP server against a real Alexa+ device

- **Task attempted:** connect the Couch Pilot MCP server (spec 2025-11-25, Streamable HTTP) to Alexa+ to drive the TV from a real Echo / Fire TV.
- **Steps taken:** read the hackathon resources (Agent Skills, Streamable HTTP transport); looked for a developer console flow to register an MCP endpoint for a personal Alexa+ account.
- **Expected:** a developer sandbox to point Alexa+ at a self-hosted MCP URL for testing.
- **Actual:** no self-serve registration outside the brands in Preview, so the end-to-end voice path can't be tested on a real device.
- **Severity:** Major
- **Workaround:** built the simulated Alexa+ path allowed by the rules: a real MCP client (official TypeScript SDK) that calls the same `/mcp` endpoint. An automated end-to-end test (`npm run test:e2e`) proves the MCP surface works.
- **Actionable suggestion:** a developer-only "MCP sandbox" in the Alexa developer console: paste an HTTPS MCP URL, link it to your own account, see the tool calls Alexa+ makes in a log.

4. Which MCP features Alexa+ supports is unclear

- **Task attempted:** design multi-turn tool calls (a clarifying question, a payment confirmation) for Alexa+.

- **Steps taken:** read the MCP 2025-11-25 spec (elicitation, structured content, tool annotations) and the hackathon pages.
- **Expected:** a support matrix: does Alexa+ honor `elicitation/create`, `structuredContent`, `outputSchema`, tool annotations such as `destructiveHint`, long-running calls, progress notifications?
- **Actual:** not documented, so we could not know whether to rely on elicitation for confirmations.
- **Severity:** Minor
- **Workaround:** tools return `status: "needs_input"` with the question and numbered options, and a separate `couchpilot_answer` tool continues. This works with any MCP client.
- **Actionable suggestion:** publish an "Alexa+ MCP client capabilities" page, with timeouts for tool calls and how Alexa+ reads structured results aloud.

5. No consent-based way to navigate other Fire TV apps

- **Task attempted:** make Couch Pilot drive any streaming app, not only its own demo catalog.
- **Steps taken:** reviewed Fire TV integration options (deep links, content launcher in the Vega video sample).
- **Expected:** a permissioned API for an assistant to operate apps on the user's behalf (open, search, play, subtitles, volume), the way accessibility services do on phones.
- **Actual:** deep links and content launch cover "open X", not multi-step navigation, subtitles or season/episode selection across apps.
- **Severity:** Major for the product, not for the demo
- **Workaround:** the demo drives its own app with the exact same primitives a remote uses (focus, press, type).
- **Actionable suggestion:** see the feature request in `PRODUCT_FEEDBACK.md`.

6. AWS credentials: role ARN without source credentials

- **Task attempted:** try an optional cloud planner on Amazon Bedrock with an IAM role ARN.
- **Steps taken:** `sts:GetCallerIdentity` and `sts:AssumeRole` from the container with the environment's default credentials.
- **Expected:** a clear path from "I have a role ARN" to a working Bedrock call from a cloud dev container.
- **Actual:** `InvalidClientTokenId`. A role ARN alone can't be used without source credentials allowed to assume it, and the error doesn't say so.
- **Severity:** Minor
- **Workaround:** dropped the cloud planner from the submission. In the Alexa+ architecture, Alexa+ is the language AI and Couch Pilot executes. The Bedrock module stays in the repo, off by default.
- **Actionable suggestion:** in STS errors and Builder Center guides, explain the "source credentials + trust policy" requirement next to `AssumeRole`, with a one-command check.